

MIPS Toolchain

MD01069 UHI Reference Manual



Copyright © Imagination Technologies Limited. All Rights Reserved.

This publication contains proprietary information which is subject to change without notice and is supplied 'as is' without warranty of any kind. Imagination Technologies, the Imagination logo, PowerVR, MIPS, Meta, Enigma and Codescape are trademarks or registered trademarks of Imagination Technologies Limited. All other logos, products, trademarks and registered trademarks are the property of their respective owners.

Filename : MIPS_Toolchain.UHI Reference Manual.docx
Version : 1.0.14 External Issue
Issue Date : 12 Feb 2015
Author : Imagination Technologies Limited

Contents

1.	Introduction	3
2.	ABI	4
2.1.	Exit	4
2.2.	Open	4
2.3.	Close	5
2.4.	Read	6
2.5.	Write	6
2.6.	Lseek	6
2.7.	Unlink	7
2.8.	Fstat	7
2.9.	Argc	8
2.10.	Argnlen	8
2.11.	Argn	9
2.12.	Plog	9
2.13.	Assert	10
2.14.	Pread	10
2.15.	Pwrite	10
2.16.	Link	11
3.	Startup and syscall handlers	12

List of Figures

Figure 1 UHI in Embedded OS	3
-----------------------------------	---

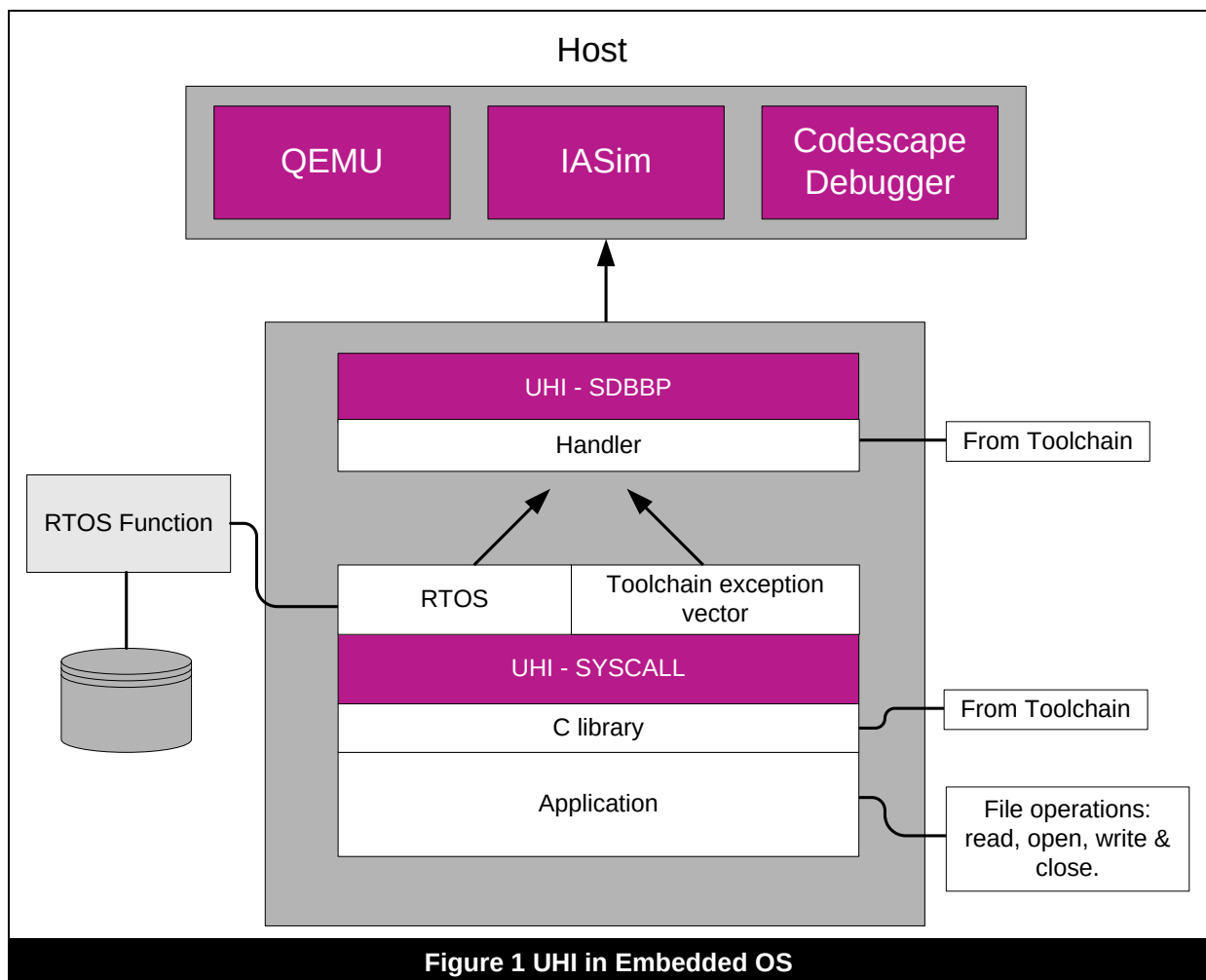
1. Introduction

A hosting interface provides a mechanism for user-mode applications to request services that require complex processing or knowledge of the current hardware. Such requests are either processed on the same target by a higher level OS (self-hosting or fully hosted) or offloaded to a remote host (semi-hosted).

The MIPS Unified Hosting Interface (UHI) aims to support both scenarios of self-hosting and semi-hosting without the need to neither modify the software when switching between the two nor require any special knowledge of the current hardware. The interface is layered to allow requests to propagate upwards through software stacks, boot monitors and debuggers.

MIPS UHI is based around the SYSCALL and SDBBP instructions along with a custom ABI and a reserved SYSCALL/SDBBP number. The SYSCALL instruction is used wherever there may be a higher level of software handling, the SDBBP instruction is used when a debugger is the only remaining option.

Note: This interface does not apply to Linux userland applications where a fully featured hosting environment is provided via a SYSCALL interface.



2. ABI

UHI defines a special ABI for argument passing in order to abstract most differences between 32-bit and 64-bit architectures. Most arguments are passed by value in registers, except some larger structures that are passed by reference (with respect to target endianness). Any such structures are precisely defined in this specification. Return values are also passed in registers.

All arguments and return values are sign or zero extended as appropriate to the full width of a register.

In:

\$2 syscall number /* i.e. 1 */
\$25 operation code
\$4-\$7 operation arguments

Out:

\$2-5 operation return values

SYSCALL 1 / SDBBP 1 are reserved for UHI. \$2 must also be set to 1 to ease software servicing. Any operation that may set errno will return the errno value as an additional result. Errno values and other target specific values are specified as those in `sim/common/nltvals.def` from the GDB repository.

Unimplemented or undefined UHI operations should not return control to the target.

2.1. Exit

Field	Description
Operation code	1
Prototype	<code>void __mips_exit (int32_t exit_code)</code>
Return	None
Arguments	\$25: Operation code 1 \$4: Exit code
Comment	Allow interception of an application exit event.

2.2. Open

Field	Description
Operation code	2
Prototype	<code>int32_t __mips_open (const char *file_name, int32_t flags, int32_t mode)</code>
Return	\$2: New file descriptor or -1 in case of an error \$3: Errno (only used on error)

Field	Description																																						
Arguments	\$25: Operation code 2																																						
	\$4: Path/file name.																																						
	\$5: One or more file creation flags. Refer to Table 1: Flags for details																																						
	\$6: Permissions to use when a new file is created. Refer to Table 2: Modes for details.																																						
	Table 1: Flags																																						
	<table><tr><th>Flag</th><th>Value</th><th>Description</th></tr><tr><td>O_RDONLY</td><td>0x0000</td><td>Read enabled</td></tr><tr><td>O_WRONLY</td><td>0x0001</td><td>Write enabled</td></tr><tr><td>O_RDWR</td><td>0x0002</td><td>Read and write enabled</td></tr><tr><td>O_APPEND</td><td>0x0008</td><td>Open in append mode</td></tr><tr><td>O_CREAT</td><td>0x0200</td><td>Create a new file if the file does not exist</td></tr><tr><td>O_TRUNC</td><td>0x0400</td><td>If the file already exists truncated to length 0</td></tr><tr><td>O_EXCL</td><td>0x0800</td><td>Generate error on open if file exists</td></tr></table>	Flag	Value	Description	O_RDONLY	0x0000	Read enabled	O_WRONLY	0x0001	Write enabled	O_RDWR	0x0002	Read and write enabled	O_APPEND	0x0008	Open in append mode	O_CREAT	0x0200	Create a new file if the file does not exist	O_TRUNC	0x0400	If the file already exists truncated to length 0	O_EXCL	0x0800	Generate error on open if file exists														
	Flag	Value	Description																																				
	O_RDONLY	0x0000	Read enabled																																				
	O_WRONLY	0x0001	Write enabled																																				
	O_RDWR	0x0002	Read and write enabled																																				
	O_APPEND	0x0008	Open in append mode																																				
	O_CREAT	0x0200	Create a new file if the file does not exist																																				
	O_TRUNC	0x0400	If the file already exists truncated to length 0																																				
	O_EXCL	0x0800	Generate error on open if file exists																																				
	Table 2: Modes																																						
<table><tr><th>Flag</th><th>Value</th><th>Description</th></tr><tr><td>S_IXOTH</td><td>0001</td><td>Others have execute permission</td></tr><tr><td>S_IWOTH</td><td>0002</td><td>Others have write permission</td></tr><tr><td>S_IROTH</td><td>0004</td><td>Others have read permission</td></tr><tr><td>S_IRWXO</td><td>0007</td><td>Others have read, write and execute permission</td></tr><tr><td>S_IXGRP</td><td>0010</td><td>Group has execute permission</td></tr><tr><td>S_IWGRP</td><td>0020</td><td>Group has write permission</td></tr><tr><td>S_IRGRP</td><td>0040</td><td>Group has read permission</td></tr><tr><td>S_IRWXG</td><td>0070</td><td>Group has read, write and execute permission</td></tr><tr><td>S_IXUSR</td><td>0100</td><td>User (file owner) has execute permission</td></tr><tr><td>S_IWUSR</td><td>0200</td><td>User has write permission</td></tr><tr><td>S_IRUSR</td><td>0400</td><td>User has read permission</td></tr><tr><td>S_IRWXU</td><td>0700</td><td>User has read, write and execute permission</td></tr></table>	Flag	Value	Description	S_IXOTH	0001	Others have execute permission	S_IWOTH	0002	Others have write permission	S_IROTH	0004	Others have read permission	S_IRWXO	0007	Others have read, write and execute permission	S_IXGRP	0010	Group has execute permission	S_IWGRP	0020	Group has write permission	S_IRGRP	0040	Group has read permission	S_IRWXG	0070	Group has read, write and execute permission	S_IXUSR	0100	User (file owner) has execute permission	S_IWUSR	0200	User has write permission	S_IRUSR	0400	User has read permission	S_IRWXU	0700	User has read, write and execute permission
Flag	Value	Description																																					
S_IXOTH	0001	Others have execute permission																																					
S_IWOTH	0002	Others have write permission																																					
S_IROTH	0004	Others have read permission																																					
S_IRWXO	0007	Others have read, write and execute permission																																					
S_IXGRP	0010	Group has execute permission																																					
S_IWGRP	0020	Group has write permission																																					
S_IRGRP	0040	Group has read permission																																					
S_IRWXG	0070	Group has read, write and execute permission																																					
S_IXUSR	0100	User (file owner) has execute permission																																					
S_IWUSR	0200	User has write permission																																					
S_IRUSR	0400	User has read permission																																					
S_IRWXU	0700	User has read, write and execute permission																																					
Comment	Special filenames: “/dev/stdin”, “/dev/stdout” and “/dev/stderr” for stdin, stdout and stderr respectively. File descriptors 0, 1 and 2 are reserved for these streams.																																						

2.3. Close

Field	Description
Operation code	3
Prototype	int32_t __mips_close (int32_t fd)

Field	Description
Return	\$2: 0 on success or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 3 \$4: File descriptor to close
Comment	

2.4. Read

Field	Description
Operation code	4
Prototype	long __mips_read (int32_t file, void *buffer, long len)
Return	\$2: Number of bytes read or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 4 \$4: File descriptor to read from \$5: Buffer to read into \$6: Max number of bytes to read
Comment	The buffer must span at least len bytes.

2.5. Write

Field	Description
Operation code	5
Prototype	long __mips_write (int32_t file, const void *buffer, long count)
Return	\$2: Number of bytes read or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 5 \$4: File descriptor to write to \$5: Buffer to write from \$6: Max number of bytes to write
Comment	The buffer must span at least count bytes.

2.6. Lseek

Field	Description
Operation code	6
Prototype	long __mips_lseek (int32_t file, long offset, int32_t whence)
Return	\$2: New offset or -1 in case of error \$3: Errno (only used on error)

Field	Description														
Arguments	\$25: Operation code 6														
	\$4: File descriptor to operate on														
	\$5: Number of bytes to reposition														
	\$6: How to reposition. Refer Table 3: Whence values for possible values.														
	<table><tr><th colspan="3">Table 3: Whence values</th></tr><tr><th>Flag</th><th>Value</th><th>Description</th></tr><tr><td>SEEK_SET</td><td>0x0001</td><td>The offset is set to 'offset' bytes</td></tr><tr><td>SEEK_CUR</td><td>0x0002</td><td>The offset is set to its current location plus 'offset' bytes</td></tr><tr><td>SEEK_END</td><td>0x0004</td><td>The offset is set to the size of the file plus 'offset' bytes</td></tr></table>	Table 3: Whence values			Flag	Value	Description	SEEK_SET	0x0001	The offset is set to 'offset' bytes	SEEK_CUR	0x0002	The offset is set to its current location plus 'offset' bytes	SEEK_END	0x0004
Table 3: Whence values															
Flag	Value	Description													
SEEK_SET	0x0001	The offset is set to 'offset' bytes													
SEEK_CUR	0x0002	The offset is set to its current location plus 'offset' bytes													
SEEK_END	0x0004	The offset is set to the size of the file plus 'offset' bytes													
Comment															

2.7. Unlink

Field	Description
Operation code	7
Prototype	int32_t __mips_unlink (const char *file)
Return	\$2: 0 on success or non-zero on failure \$3: Errno (only used on error)
Arguments	\$25: Operation code 7 \$4: Path/file name to remove from file system
Comment	

2.8. Fstat

Field	Description
Operation code	8
Prototype	int32_t __mips_fstat (int32_t file, struct uhi_stat *sbuf)
Return	\$2: 0 on success or non-zero on failure \$3: Errno (only used on error)

Field	Description
Arguments	\$25: Operation code 8 \$4: File descriptor to operate on \$5: Buffer to store the status information. The stat structure is given below: <pre> struct uhi_stat { int16_t st_dev; uint16_t st_ino; uint32_t st_mode; uint16_t st_nlink; uint16_t st_uid; uint16_t st_gid; int16_t st_rdev; uint64_t st_size; uint64_t st_atime; uint64_t st_spare1; uint64_t st_mtime; uint64_t st_spare2; uint64_t st_ctime; uint64_t st_spare3; uint64_t st_blksize; uint64_t st_blocks; uint64_t st_spare4[2]; }; </pre>
Comment	The UHI stat structure is abstracted to be the same regardless of the target ABI. The upper-32 bits of several fields should be discarded for a 32-bit program.

2.9. Argc

Returns the number of arguments. Use of this and the following two operations must be optional to allow executing in a non-hosted environment. See 'Startup and syscall handlers' on page 12 for argument handling rules.

Field	Description
Operation code	9
Prototype	int32_t __mips_argc (void)
Return	\$2: Number of arguments
Arguments	\$25: Operation code 9
Comment	Returns the number of arguments, minimum of 1.

2.10. Argnlen

Field	Description
-------	-------------

Field	Description
Operation code	10
Prototype	int32_t __mips_arglen (int32_t n)
Return	\$2: Length of n'th argument or -1 on error
Arguments	\$25: Operation code 10 \$4: Argument number
Comment	Returns the length of the n'th argument (excluding the terminating null character).

2.11. Argn

Field	Description
Operation code	11
Prototype	int32_t __mips_argn (int32_t n, char *buf)
Return	\$2: Return 0 on success or -1
Arguments	\$25: Operation code 11 \$4: Argument number \$5: Destination buffer, where the argument is copied
Comment	Returns the n'th argument in the buffer provided. The buffer must be at least as large as the value obtained from argnlen for same argument.

These operations can be used to construct argv and argc as shown in the following pseudo-code:

```
int i, argc;
char **argv;

argc = __mips_argc ();
argv = malloc (argc * sizeof (char*));

for (i = 0; i < argc; i++)
{
    argv[i] = malloc (__mips_arglen (i) + 1);
    __mips_argn (i, argv[i]);
}

main (argc, argv);
```

2.12. Plog

Field	Description
Operation code	13
Prototype	int32_t __mips_plog (const char* msg, int32_t num)
Return	\$2: Number of characters output
Arguments	\$25: Operation code 13 \$4: Null terminated character array to output \$5: Integer to use in formatting msg (if %d is present)
Comment	A limited form of printf. Initially support for just one integer argument. String formatting to be done by the hosting platform.

2.13. Assert

Field	Description
Operation code	14
Prototype	void __mips_assert (char *message, char *filename, int line_num)
Return	None
Arguments	\$25: Operation code 14 \$4: Message to display \$5: File containing assert statement \$6: Line number in the specified file
Comment	Indicates a software assertion has triggered.

2.14. Pread

Field	Description
Operation code	19
Prototype	long __mips_pread (int32_t fd, void *buf, long count, long offset)
Return	\$2: Number of bytes read or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 19 \$4: File descriptor to read from \$5: Buffer to read into \$6: Max number of bytes to read \$7: Offset in file from which bytes to be read
Comment	

2.15. Pwrite

Field	Description
Operation code	20
Prototype	long __mips_pwrite (int32_t fd, const void *buf, long count, long offset)
Return	\$2: Number of bytes written or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 20 \$4: File descriptor to write to \$5: Buffer to write from \$6: Max number of bytes to write \$7: Offset in file at which bytes to be written
Comment	

2.16. Link

Field	Description
Operation code	22
Prototype	int32_t __mips_link (const char *oldname, const char *newname)
Return	\$2: 0 on success or -1 in case of error \$3: Errno (only used on error)
Arguments	\$25: Operation code 22 \$4: Old name of a file \$5: New name to be given
Comment	

3. Startup and syscall handlers

A UHI compliant CRT will expect to receive a number of parameters set up before control is passed to the entry point. These parameters are:

\$4 == argc

\$5 == argv – an array of char* pointers

\$6 == envp – a null terminated array of char* pointers with each string having format key=value

\$7 == reserved – because it has had multiple meanings in the past.

When \$4 is 0 all other parameters will be ignored.

When \$4 is -1 the UHI interface for obtaining argc and argv will be used during startup to obtain the command line.

When \$4 has any other positive value then it represents argc and the remaining parameters become valid.

All other negative values provided for \$4 are reserved for future extension.